COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/7/26	**DATE SUBMITTED:**	10/7/26
NAME:	Arididdon, Gabriel Dwyane D. Esguerra, Paula G.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis	
Given the above information, the total number of comparisons in the worst case is: O(n^2).	
Graph Analysis	Arididdon: As the array size n increases, the number of operations also increases at an accelerating rate. The worst time complexity for this is O(n^2)
	Esguerra: The scale is not measured in linear, where n size increases by 10. So as the input size doubles, the operation increases. The Worst complexity is at $O(n^2)$.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student
{
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v)
    {
    }
    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student &p) const
    {
        return this->name == p.name;
    }
    // The ordering of a set of people is defined by their name
    bool operator<(const Student &p) const
    {
        return this->name < p.name;
    }
    bool operator>(const Student &p) const
    {
        return this->name > p.name;
```

```
    }
};

auto generate_random_Student(int max)
{
    std::random_device rd;
    std::mt19937 rand(rd());
    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type>
uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand),
uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student> &people)
{
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;

        else if (mid_element == P && mid_element.is_vaccinated() ==
true)
            return false;
```

```
        else if (mid_element > P)
            std::advance(last, -mid_element_index);

        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be
vaccinated
        if (range_length == 1)
            return true;
    }
}

void search_test(int size, Student p)
{
    std::vector<Student> people;
    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
```

```
                    << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
                    << ") " << "does not need vaccination." <<
std::endl;
}

int main()
{
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Output

Arididion

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	1 us	Time taken to search = 1 microseconds Student (489 860) needs vaccination.	Since the input size is only 1000, the execution speed is only 1 us
10000	2 us	Time taken to search = 2 microseconds Student (489 860) needs vaccination.	It takes 2 us in handling input size 10000
100000	3 us	Time taken to search = 3 microseconds Student (489 860) needs vaccination.	For 100000, it takes 3 us

Esguerra

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	1us	Time taken to search = 1 microseconds Student (418 287) needs vaccination.	The search test only requires 1000, in 1000 generated students, so the time execution is at 1 microseconds.
10000	1us	Time taken to search = 1 microseconds Student (418 287) needs vaccination.	This is similar to 1000 search test, the time execution is at 1 microseconds
100000	2us	Time taken to search = 2 microseconds Student (418 287) needs vaccination.	Then since the number os search test is bigger than the rest, the time execution is at 2 microseconds.

Analysis

Aridido: After initializing for many times and inputting smaller or larger inputs, I concluded that when the input size increases, the execution time also increases. However, since it is a random student, the time taken also changes on every initialization.

Esguerra: In trying to experiment with the numbers, it depends on the machine and time execution whether it will be different, sometimes even the number is similar to the highest, the time execution is still different, it can be only at 1us or 2us, very similar to different.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis &
ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1

```cpp
#include <iostream>
#include <vector>
#include <chrono>

// returns true if all elements in A are unique
bool Unique(std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j])
                return false;
        }
    }
    return true;
}

// Worst case: all elements unique, so every pair is compared -> n(n-1)/2 comparisons
void worst_case_test(int size) {
    std::vector<int> A(size);
    for (int i = 0; i < size; i++)
        A[i] = i + 1;

    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool result = Unique(A);
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Worst case, n = " << size << std::endl;
```

```
    std::cout << "Time taken = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count()
              << " microseconds" << std::endl;
    std::cout << "Unique returned " << result << std::endl << std::endl;
}

// Best case: A[0] == A[1], so Unique stops after 1 comparison
void best_case_test(int size) {
    std::vector<int> A(size);
    for (int i = 0; i < size; i++)
        A[i] = i + 1;
    A[1] = A[0];

    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
    bool result = Unique(A);
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();

    std::cout << "Best case, n = " << size << std::endl;
    std::cout << "Time taken = "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(end -
begin).count()
              << " nanoseconds" << std::endl;
    std::cout << "Unique returned " << result << std::endl << std::endl;
}

int main() {
    worst_case_test(1000);
    worst_case_test(10000);
    worst_case_test(100000);

    best_case_test(1000);
    best_case_test(10000);
    best_case_test(100000);
```

```
    return 0;
}
```

	Aridido n	Esguerra
Theoretical Analysis	T(N) = n(n-1) / 2 since it compares each element only with the remaining elements after it to avoid checking items against themselves of repeating previous checks.The worst case is O(n^2) since it is nested loop.	The outer loop runs n times from 0 to n-1, on the other hand, the inner loop starts at i + 1 until n -1.The worst case analysis is $O(n^2)$, this occurs when all elements in the array are unique. While the best case is when the code terminates at the first two elements in the array are duplicates.
Experimental Analysis	Worst case, n = 1000 Time taken = 3001 microseconds Unique returned 1 Worst case, n = 10000 Time taken = 246960 microseconds Unique returned 1 Worst case, n = 100000 Time taken = 24683816 microseconds Unique returned 1 Best case, n = 1000 Time taken = 0 nanoseconds Unique returned 0 Best case, n = 10000 Time taken = 0 nanoseconds Unique returned 0 Best case, n = 100000 Time taken = 0 nanoseconds Unique returned 0	Worst case, n = 1000 Time taken = 4043 microseconds Unique returned 1 Worst case, n = 10000 Time taken = 407426 microseconds Unique returned 1 Worst case, n = 100000 Time taken = 40169311 microseconds Unique returned 1 Best case, n = 1000 Time taken = 200 nanoseconds Unique returned 0 Best case, n = 10000 Time taken = 100 nanoseconds Unique returned 0 Best case, n = 100000 Time taken = 100 nanoseconds Unique returned 0
Analysis and Comparison	The experimental result directly confirm the theory as increasing n by a factor of 10 increases its execution time. The best case is in 0 nanoseconds since the algorithm immediately terminates on the first comparison.	At 1000 elements the best case is the one comparison and the worst case occurs without a matching pair to break out early, this crreates an algorithm to implement a quadratic groth.

Problem 2

```cpp
#include <iostream>
#include <chrono>

typedef unsigned long long ull;

const int RUNS_R = 1000;
const int RUNS_B = 1000000;

// O(n): n recursive calls
ull rpower(ull x, int n) {
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

// O(log n): n is halved every call
ull brpower(ull x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        ull y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }
    else {
        ull y = brpower(x, n / 2);
        return y * y;
    }
}

// times both algorithms for the same n
void power_test(int n) {
    volatile ull result;

    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
    for (int r = 0; r < RUNS_R; r++)
```

```
        result = rpower(2, n);
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();
    long long time_r =
std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() /
RUNS_R;

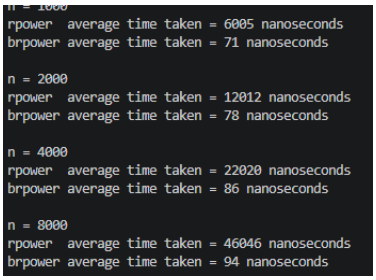
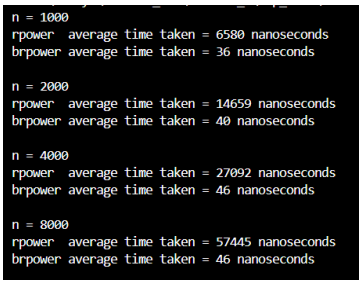
    begin = std::chrono::steady_clock::now();
    for (int r = 0; r < RUNS_B; r++)
        result = brpower(2, n);
    end = std::chrono::steady_clock::now();
    long long time_b =
std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() /
RUNS_B;

    std::cout << "n = " << n << std::endl;
    std::cout << "rpower  average time taken = " << time_r << " nanoseconds"
<< std::endl;
    std::cout << "brpower average time taken = " << time_b << " nanoseconds"
<< std::endl << std::endl;
}



int main() {
    power_test(1000);
    power_test(2000);
    power_test(4000);
    power_test(8000);

    return 0;
}
```

	Arididdon	Esguerra
Theoretical Analysis	For rpower, T(n) = 2n + 1 or O(n) because each step decreases n by 1 For brpower, n is halved every call so it is O(log n) and it is faster than rpower since it reduce operations exponentially.	This code uses two approaches, a linear recursive method (x^2) and a logarithmic divide and conquer method. The Linear Recursion uses T(n) - T(n -1) + O(1) at a O(n) time complexity. While, the logarithmic divide and conquer uses T(n) = ([n/2]) + O(1), at O(logn) time complexity.
Experimental Analysis		
Analysis and Comparison	The experimental result directly reflect on the theoretical complexity differences between rpower and brpower. As n doubles from 1000 to 8000, the execution time increases linearly since it has recursive calls. On the other hand, brpower maintains a nearly average time even when the n doubles because it halves n at each recursion.	The main function creates four values of n, this causes the values to double. When n doubles, the number of operations grow also, in Linear Recursive. Then on Logarithmic divide and conquer, the operations only increases only by 1 element higher.

Conclusion	**Arididdon:** Through this activity, I learned how to evaluate algorithms using theoretical analysis and learned to use timing functions for smaller to bigger n values and observe different growth rates in O(n), O(log n), and O(n^2).
	Esguerra: In this laboratory experiment I have learned to analysis my code through the use of searching, where the linear approach and divide and conquer strategies are a growth rate used in the program. The program still depends on how the code is created, while compiler settings and fast processors can speed up execution, but your code has a poor algorithm, it will still execute slowly. Furthermore, linear recursion has to handle element growth(called linear growth), then divide and conquer has to deal with a single additional operation, this is called a logarithmic growth. Lastly, I have learned how they

	show their final output, it takes a while to finish, and it depends on the code optimization, compiler settings and good processors. It really is critical to know what to use and how to use the things we will be using in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explaining concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini to explain hard concepts and also with some code assistance in supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.